Designing and Solving Real-World Synchronization Problems

Objective

The goal of this assignment is to design, analyze, and solve practical synchronization problems that can occur in real-world systems. By identifying the challenges and applying appropriate synchronization techniques, we aim to maintain system consistency, avoid conflicts, and optimize performance. The solutions will involve the use of semaphores, mutexes, monitors, and techniques to prevent deadlock and race conditions.

Problem 1: Smart Parking Garage System

Problem Design:

A smart parking garage system monitors available parking spaces for vehicles entering and exiting. Multiple sensors detect car entries and exits simultaneously, but there are limited spots, and the system must keep track of the availability in real-time. The system needs to ensure that updates to the number of available spots are handled correctly to avoid errors.

Conflicting Operations:

- **Concurrent Entries**: Two vehicles could try to enter the garage and claim the same parking spot, causing an inaccurate count.

- **Simultaneous Exits**: Multiple cars exiting at once might cause the system to double-count the freed spaces.

Potential Issues:

- **Race Condition**: Two sensors accessing and modifying the available parking spots at the same time could result in incorrect counts.

- **Deadlock**: If sensors acquire locks on resources in a conflicting order, they may end up waiting indefinitely for each other.

- **Starvation**: If sensors keep updating counts for one type of operation (e.g., entries) without giving access to other operations (e.g., exits), some requests might not get processed in a timely manner.

Synchronization Solution:

To handle the conflicting operations efficiently, we use semaphores and mutexes:

- **Semaphore**: We will initialize a semaphore, `parking_spots`, with the number of available spots. The semaphore will ensure that when a car tries to enter, it only proceeds if there are available spots.

- **Mutex**: A mutex will be used to protect the critical section where the spot count is updated. This ensures that only one sensor can modify the count at a time.

Code Example:

```
wait(parking_spots); // Decrease available spots

acquire(mutex); // Enter critical section to update the count

update_spot_count(--); // Car enters

release(mutex); // Exit critical section
```

- Similarly, when a car exits:

```
acquire(mutex); // Enter critical section to update the count

update_spot_count(++);
```

release(mutex); // Exit critical section

signal(parking_spots); // Increase available spots

Deadlock Prevention:

- **Minimal Critical Section**: Ensure that the critical section is as short as possible, allowing other threads to access the parking system quickly.

- **Avoid Nested Locking**: Prevent sensors from acquiring multiple locks that might lead to deadlock by enforcing a clear lock acquisition order.

Problem 2: Online Food Delivery System (Order Queue)

Problem Design:

In an online food delivery app, multiple delivery drivers (consumers) pick up orders from a shared order list, while customers (producers) place new orders. There is a need for synchronization to ensure that drivers don't pick up the same order, and that new orders are properly placed in the queue.

Conflicting Operations:

- A driver might try to pick up an order while another driver is also attempting to pick the same order.

- Multiple customers can place orders simultaneously, but without proper synchronization, the system may not correctly handle the order queue.

Potential Issues:

- **Race Condition**: Two drivers accessing and modifying the order list simultaneously can cause incorrect order assignments.

- **Deadlock**: If both drivers and customers acquire locks in an improper order, a deadlock would occur, where no one can process orders.

- **Starvation**: New orders might get delayed if the system keeps prioritizing certain drivers or if the queue becomes too congested.

Synchronization Solution:

We can implement a bounded-buffer solution to handle the producer-consumer relationship between customers and drivers.

- **Semaphore**:
    - `empty` to track the available order slots.
    - `full` to track the number of orders in the queue.
- **Mutex** to control access to the shared order queue.

Code Example:

- **Customer (Producer)** places orders:

```
wait(empty); // Wait for space in the queue
wait(mutex); // Enter critical section
place_order(); // Add order to the queue
signal(mutex); // Exit critical section
signal(full); // Signal that an order is available for drivers
```

- **Driver (Consumer)** picks up orders:

```
wait(full); // Wait for available orders
wait(mutex); // Enter critical section
pick_order(); // Pick up an order
```

signal(mutex); // Exit critical section

signal(empty); // Signal that there is space for more orders

Deadlock Avoidance:

- By using semaphores to track the order of operations (i.e., ensuring that orders are placed before being picked up), and ensuring that the mutex is used only when necessary, we avoid circular dependencies.

Problem 3: University Library Booking System

Problem Design:

In a university library system, multiple students book study rooms through an online portal. There are a limited number of rooms, and at the same time, the admin might reserve some rooms for maintenance. We need to ensure that room bookings are synchronized so that multiple students don't book the same room and admins can perform maintenance without interfering with active bookings.

Conflicting Operations:

- A student might try to book a room while an admin is marking it unavailable.
- Multiple students might attempt to book the same room at the same time.

Potential Issues:

- **Race Condition**: Two students being assigned the same room simultaneously.
- **Deadlock**: Admins and students waiting on each other to release locks on room status, causing both operations to halt.
- **Starvation**: Students might never be able to book rooms if admin reservations frequently block access.

Synchronization Solution:

We will use a monitor to encapsulate the booking operations, ensuring that only one process (admin or student) modifies the room availability at any time. A monitor can handle synchronization for both the booking and maintenance tasks.

- **Monitor Structure**:
    - **Condition Variables**: Admins and students wait for the available room, and when a room is freed, the corresponding process can proceed.

Code Example:

```
monitor RoomBooking {
    boolean rooms[N]; // Array indicating room availability
    condition available_rooms; // Condition variable to wait for available rooms

    procedure book_room() {
        if (room_available) {
            allocate_room();
        } else {
            wait(available_rooms); // Wait until room is available
        }
    }

    procedure block_room() {
        mark_room_for_maintenance();
        signal(available_rooms); // Notify that room status is updated
    }
```

}

Deadlock Prevention:

- **Monitor Guarantees**: Only one process (admin or student) can modify room status at a time, preventing any race conditions or deadlocks.

- **Starvation Avoidance**: By using signals to notify waiting processes, we ensure that no process is indefinitely blocked.

Conclusion :

In all three problems, applying appropriate synchronization mechanisms such as semaphores, mutexes, and monitors ensures data consistency, fairness, and efficient operation. These techniques are crucial in real-world applications to avoid race conditions, deadlocks, and starvation, ensuring smooth operation and fairness in shared resource management.